

BrailleCodeCompiler

23PT01 - Aakash Velusamy | 23PT11 - Harshil Bhavik Momaya

PROBLEM OVERVIEW:

Programming languages are written in plain text using letters, numbers, and symbols visible on the display. The compiler always expects standard text input and cannot process Braille directly. This project introduces a compiler that starts with Braille as the native source code, treating it as the true programming language rather than just an accessibility output.

LANGUAGE BASICS:

It is designed as a simple version of Python to keep things clear and approachable. It supports basic features such as creating variables, performing math operations, making decisions with if-else statements, looping with while loops, and print statements. Each keyword, operator, and identifier is represented by a specific six-dot Braille Unicode pattern. The scope in the program is defined using indentation with Braille spaces exactly like how python does.

WORKING PRINCIPLE:

The compiler follows a clear sequence of steps. First, it scans the Braille characters and turns them into meaningful tokens such as keywords, numbers, and operators. Next, it checks whether these tokens follow the language rules and builds an Abstract Syntax Tree that represents the structure of the program. Then it verifies that variables are declared and used correctly, tracks their scope, and performs basic checks using a symbol table. Finally, a custom interpreter walks through this tree, runs the program logic, and produces the output.

DEVELOPMENT INTERFACE:

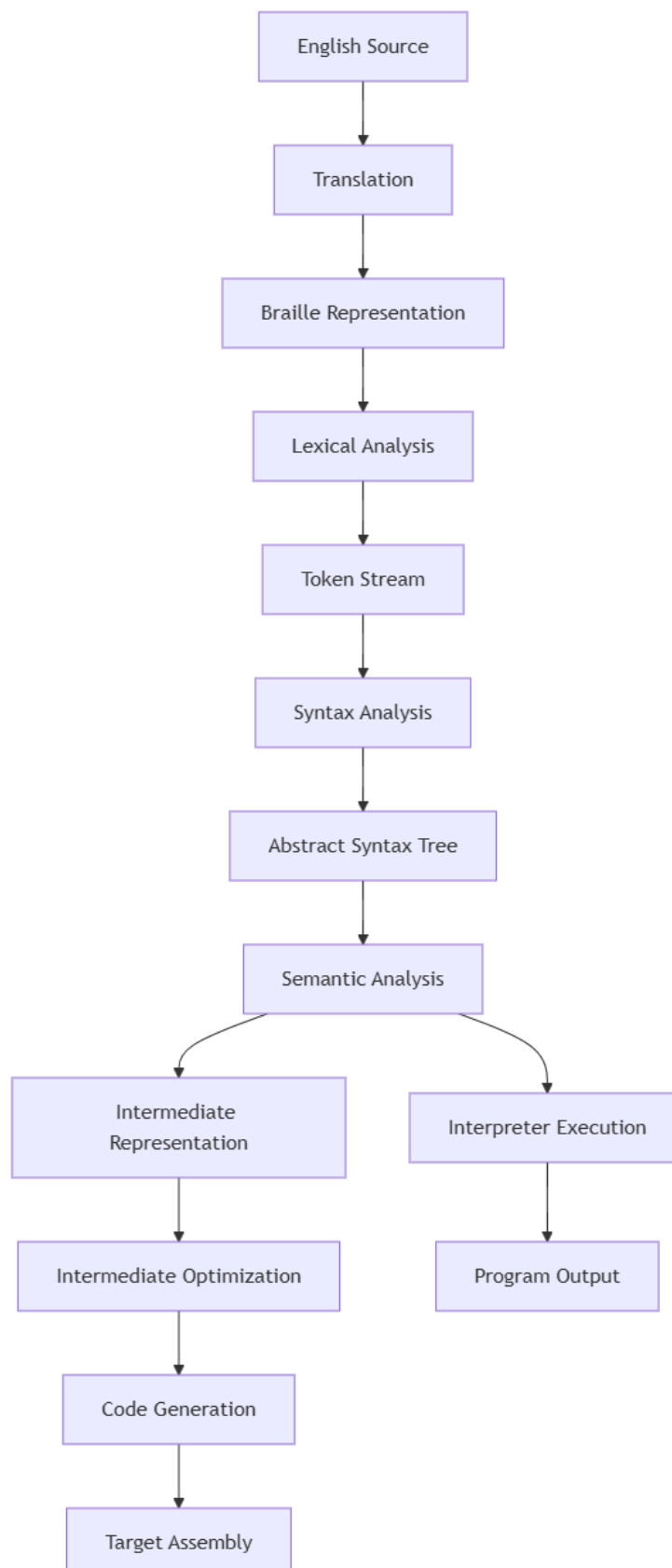
To make testing and showing the project easier, users can write code in familiar English-like syntax. The system automatically converts this input into Braille before starting the compilation process. When the program finishes running, the results are displayed in plain English text so everyone can read and understand them quickly.

CONCLUSION:

Overall, BrailleCode Compiler is not meant to replace that practical method and purely exists for academic exploration and to gain a deeper understanding of how compilers are built, and to demonstrate how classical principles can adapt to new methods of code representation.

System Architecture:

The following flowchart represents the technical progression of the compilation units through the various stages of the compiler middle-end and back-end.



Compilation Process:

Source Translation: The system processes English source strings into equivalent Braille Unicode characters based on Grade 1 Braille standards. This bidirectional mapping ensures that the subsequent scanner operates exclusively on tactile Unicode patterns.

Lexical Analysis: The scanner partitions the input stream into a discrete sequence of tokens based on defined regular languages. This process utilizes a stack based algorithm to track indentation depths and emit structural tokens for scope management.

Syntax Analysis: A recursive descent parser constructs a hierarchical abstract syntax tree from the token stream. The parser enforces a context free grammar with seven distinct levels of operator precedence to ensure correct expression binding.

Semantic Analysis: The analyzer performs a traversal of the tree to enforce language constraints and maintain symbol tables. It conducts type inference and identifies undeclared variable references across nested lexical scopes.

Intermediate Representation: The system lowers the validated tree into a machine independent three address code format. This representation linearizes the program flow and facilitates the implementation of optimization algorithms.

Intermediate Optimization: The middle-end executes constant folding and dead code elimination on the intermediate representation. These passes refine the instruction sequence to improve runtime efficiency and reduce the final memory footprint.

Target Code Generation: The back-end translates optimized intermediate instructions into pseudo-assembly for a virtual machine. A linear scan allocator manages register assignments across the eight available registers and handles memory spilling when register pressure is high.

Runtime Execution: The interpreter performs a tree walk evaluation of the final program state. It maintains a runtime environment for variable storage and captures output streams while enforcing safety limits on loop iterations.

23XTE7 - Compiler Design Lab

BrailleCodeCompiler

Lexical Analysis Syntax Analysis Semantic Analysis Intermediate Code Generation Code Optimization Code Generation

Compile & Run Debug Examples Clear

EXPORT .bcc .brt AST Output Full Report

SOURCE.BCC

```

1 x = 10
2 y = 20
3 z = x + y * 2
4
5 if z > 30:
6     print(z)
7     z = z - 1
8 elif z == 30:
9     print(0)
10 else:
11     print(1)
12
13 while x > 0:
14     x = x - 1
15
16 print(x)
17 print(z)
```

RESULTS

Output Lexical Analysis Syntax Analysis Semantic Analysis Intermediate Code Generated Passed

- Keyword
- Operator
- Literal
- Identifier
- Punctuation
- Structure

L1	IDENTIFIER x	=	INTEGER 10	NEWLINE
L2	IDENTIFIER y	=	INTEGER 20	NEWLINE
L3	IDENTIFIER z	=	IDENTIFIER x	+ IDENTIFIER y * INTEGER 2 NEWLINE
L5	IF if	IDENTIFIER z	>	INTEGER 30 COLON NEWLINE
L6	INDENT 0	PRINT print	(IDENTIFIER z)	RPAREN NEWLINE
L7	IDENTIFIER z	=	IDENTIFIER z	- MINUS INTEGER 1 NEWLINE
L8	DEDENT 0	ELIF elif	IDENTIFIER z	= EQUAL INTEGER 30 COLON NEWLINE
L9	INDENT 0	PRINT print	(IDENTIFIER z)	RPAREN NEWLINE
L10	DEDENT 0	ELSE else	COLON NEWLINE	
L11	INDENT 0	PRINT print	(IDENTIFIER z)	RPAREN NEWLINE
L13	DEDENT 0	WHILE while	IDENTIFIER x	>
L14	INDENT 0	IDENTIFIER x	=	IDENTIFIER x
L16	DEDENT 0	PRINT print	(IDENTIFIER x)	RPAREN NEWLINE
L17	PRINT print	(IDENTIFIER z)	RPAREN NEWLINE	

⠠ BRAILLE

23XT67 - Compiler Design Lab

BrailleCodeCompiler

Lexical Analysis Syntax Analysis Semantic Analysis Intermediate Code Generation Code Optimization Code Generation

Compile & Run Debug Examples Clear

EXPORT .bcc .brt AST Output Full Report

SOURCE.BCC

```

1 x = 10
2 y = 20
3 z = x + y * 2
4
5 if z > 30:
6     print(z)
7     z = z - 1
8 elif z == 30:
9     print(0)
10 else:
11     print(1)
12
13 while x > 0:
14     x = x - 1
15
16 print(x)
17 print(z)
```

RESULTS

Output Lexical Analysis Syntax Analysis Semantic Analysis Intermediate Code G Passed

```

1 50
2 0
3 49
```

FINAL VARIABLES

NAME	VALUE	TYPE
x	0	int
y	20	int
z	49	int

BRILLE

23XT07 - Compiler Design Lab

23PT01 Aakash Velusamy 23PT11 Harshil Bhavik Momaya

BrailleCodeCompiler

[Compile & Run](#) [Debug](#) [Examples](#) [Clear](#)

EXPORT .bcc .brl AST Output Full Report

SOURCE.BCC

```

1 x = 10
2 y = 20
3 z = x + y * 2
4
5 if z > 30:
6     print(z)
7     z = z - 1
8 elif z == 30:
9     print(0)
10 else:
11     print(1)
12
13 while x > 0:
14     x = x - 1
15
16 print(x)
17 print(z)
    
```

BRILLE

RESULTS

- Statement
- Operator
- Literal

Program Program :1

- x = Assign :1
 - 10 Number :1
- y = Assign :2
 - 20 Number :2
- z = Assign :3
 - + BinOp :3
 - x Identifier :3
 - * BinOp :3
- if # :4
 - condition Label :5
 - > Compare :5
 - body Label :5
 - print Print :6
 - z = Assign :7
 - elif Label :8
 - condition Label :8
 - body Label :8
 - else Label :5
 - print Print :11
- while While :13
 - condition Label :13
 - > Compare :13
 - body Label :13

23XT07 - Compiler Design Lab

BrailleCodeCompiler

Lexical Analysis Syntax Analysis Semantic Analysis Intermediate Code Generation Code Optimization Code Generation

Compile & Run Debug Examples Clear

EXPORT .bcc .brt AST Output Full Report

SOURCE.BCC

```

1 x = 10
2 y = 20
3 z = x + y * 2
4
5 if z > 30:
6     print(z)
7     z = z - 1
8 elif z == 30:
9     print(0)
10 else:
11     print(1)
12
13 while x > 0:
14     x = x - 1
15
16 print(x)
17 print(z)
    
```

RESULTS Output Lexical Analysis Syntax Analysis Semantic Analysis Intermediate Code G Passed

NAME	TYPE	SCOPE	DECLARED
x	int	global	Line 1
y	int	global	Line 2
z	int	global	Line 3

BRILLE

23XT67 - Compiler Design Lab

23PT01Aakash Velusamy23PT11Harshil Bhavik Momaya

BrailleCodeCompiler

Compile & RunDebugExamplesClear

EXPORT .bcc .brl AST OutputFull Report

SOURCE.BCC

```
1 x = 10
2 y = 20
3 z = x + y * 2
4
5 if z > 30:
6     print(z)
7     z = z - 1
8 elif z == 30:
9     print(0)
10 else:
11     print(1)
12
13 while x > 0:
14     x = x - 1
15
16 print(x)
17 print(z)
```

BRILLE

RESULTS

OutputLexical AnalysisSyntax AnalysisSemantic AnalysisIntermediate Code GenerationCode OptimizationCode Generation

34 INSTRUCTIONS

VisualText

0 = x = 10L1; x = 10

1 = y = 20L2; y = 20

2 + t0 = y * 2L3

3 + t1 = x + t0L3

4 = z = t1L3; z = t1

5 > t2 = z > 30L5

6 IF_FALSE L0 = t2L5; if-test

7 PARAM zL6

8 CALL print 1L6; print(z)

9 - t3 = z - 1L7

10 = z = t3L7; z = t3

11 GOTO L2L5

12 L0:

13 = t4 = z = 30L8

14 IF_FALSE L1 = t4L8; if-test

15 PARAM 0L9

16 CALL print 1L9; print(0)

17 GOTO L2L8

18 L1:

19 PARAM 1L11

23XT67 - Compiler Design Lab

23PT01Aakash Velusamy23PT11Harshil Bhavik Momaya

BrailleCodeCompiler

Compile & RunDebugExamplesClear

EXPORT .bcc .brl AST OutputFull Report

SOURCE.BCC

```
1 x = 10
2 y = 20
3 z = x + y * 2
4
5 if z > 30:
6     print(z)
7     z = z - 1
8 elif z == 30:
9     print(0)
10 else:
11     print(1)
12
13 while x > 0:
14     x = x - 1
15
16 print(x)
17 print(z)
```

BRILLE

RESULTS

ysisSemantic AnalysisIntermediate Code GenerationCode OptimizationCode Generat

34 BEFORE → 34 AFTER

Optimization Log (6)Optimized IR

CONSTANT PROPAGATION#2

BEFORE

AFTER

t0 = y * 2 → t0 = 20 * 2

Replaced variable with its known constant value

CONSTANT PROPAGATION#3

BEFORE

AFTER

t1 = x + t0 → t1 = 10 + t0

Replaced variable with its known constant value

CONSTANT PROPAGATION#23

BEFORE

AFTER

t5 = x > 0 → t5 = 10 > 0

Replaced variable with its known constant value

CONSTANT PROPAGATION#25

BEFORE

AFTER

